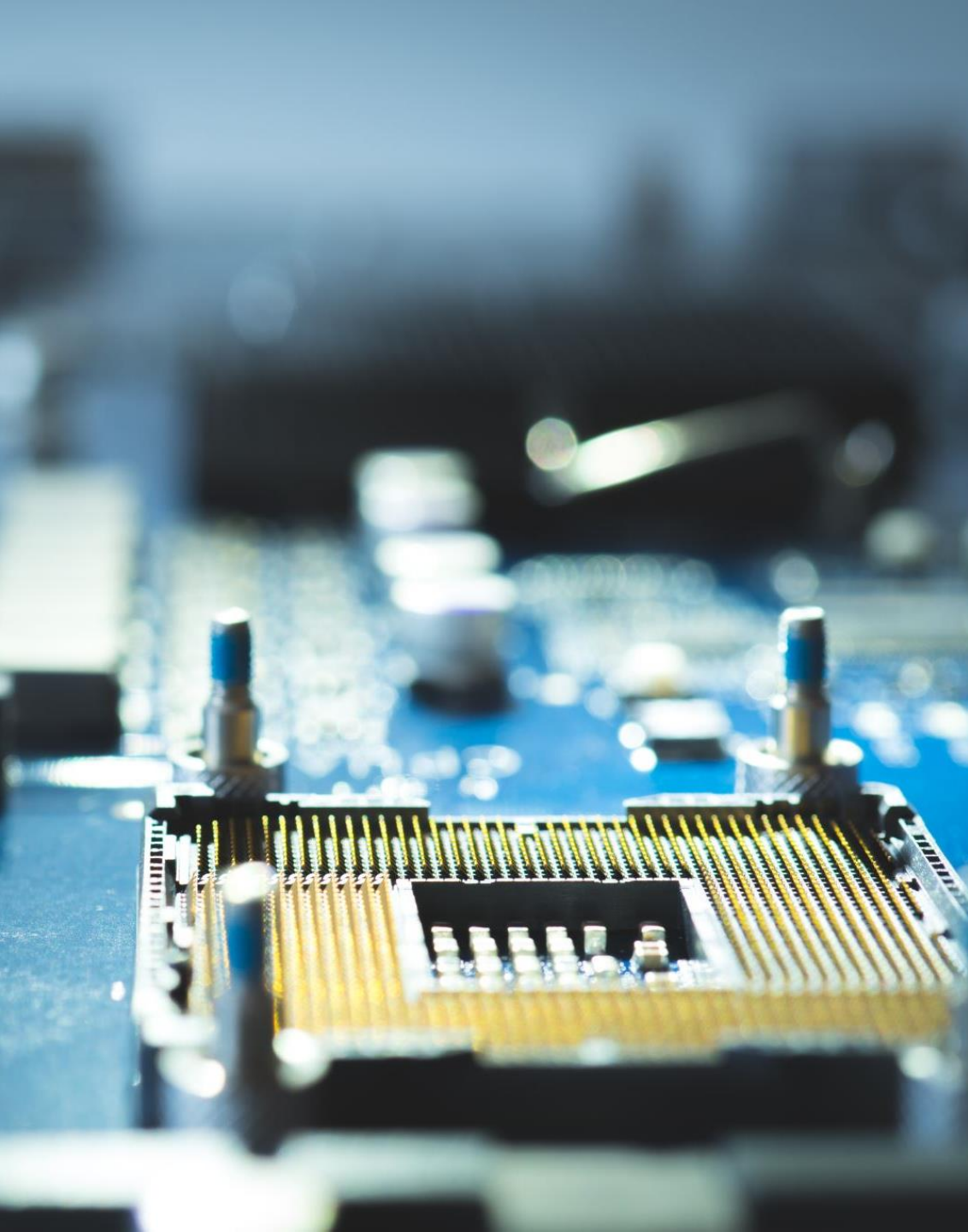




DevOps

The Technological Practices of Integrating
Security, Change Management, and Compliance



Goals

- Create and integrate security controls into the daily work
- Automate as much as possible into the deployment pipeline
- Augment manual practices, acceptances, and approvals with automated controls

Mechanisms



Make security part of everyone's job



Integrate preventive controls into the shared source code repository



Integrate security with telemetry to better enable detection and recovery



Protect deployment pipeline



Integrate deployment activities with change approval processes



Reduce reliance on separation of duty



Information Security is
Everyone's Job Every Day

Integrate Security
into Development
Iteration
Demonstrations

Have feature teams engaged
with Infosec as early as
possible



Infosec

better
understand the
team goal

Observe
implementation
as being built

Provide
guidance at
early stages



Integrate Security into Defect Tracking and Post-mortem



Track all open security issues like any work item

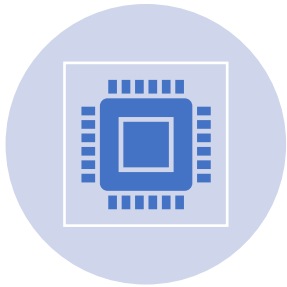


Security items can be priorities against all items



Security issues should be followed by a post-mortem to better educate engineers on how to prevent it

Integrate Preventive Security Controls into Shared Source Code Repositories and Shared Services



Add to the repository any mechanism or tool that helps ensuring security of application and environment



Libraries that are “pre-blessed” by security to fulfill specific Infosec objectives



Enables collaborations to create and operate shared-relevant platforms



The use of pre-defined libraries removes the need to have separate security design review

Integrate Security into Deployment Pipeline



Automate as many of information security tests



Static code analysis



Vulnerable dependencies



Virus scanning



Dynamic testing



Source code integrity and code signing



Tools (SonarQube, Mend, Gauntlt, OWASP Zap, Nmap, Metasploit)

Ensure Security of the Application

Focus	Write	Follow
Focus testing on bad paths	Write code to prevent abuse (rate limits, graying out submit button)	Follow OWASP guidance

Twitter example



Prevent security mistakes from being repeated (automate the checks)



Integrate security objectives into existing developer tools (provide useful information on how to fix)



Preserve Trust of Development (avoid false positives)



Maintain fast flow through Infosec through automation (automate the finding of vulnerabilities, reports sent to devs)



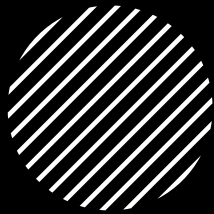
Make everything security related self-service



Take a holistic approach to achieving Infosec objectives (source code, environment, customer)



Ensure Security of Software Supply Chain

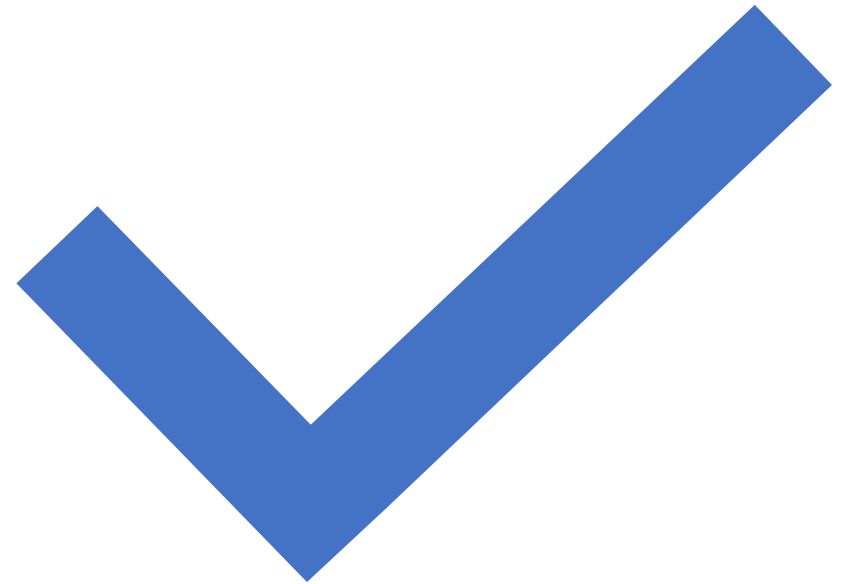


“We are no longer writing customized software – instead, we assemble what we need from open-source parts, which has become the software supply chain that we are very much reliant upon” – Josh Corman

Ensure Security of the Environment



- Put monitoring controls to ensure that all production instances match the known good states
- Automated test to check that all appropriate settings have been correctly applied (e.g., open ports)
- Scan environment for known vulnerabilities (e.g., SQL injection)



Integrate Information Security into Production Telemetry



Integrate security telemetry into the same tools used by Devs and Qas



Application telemetry

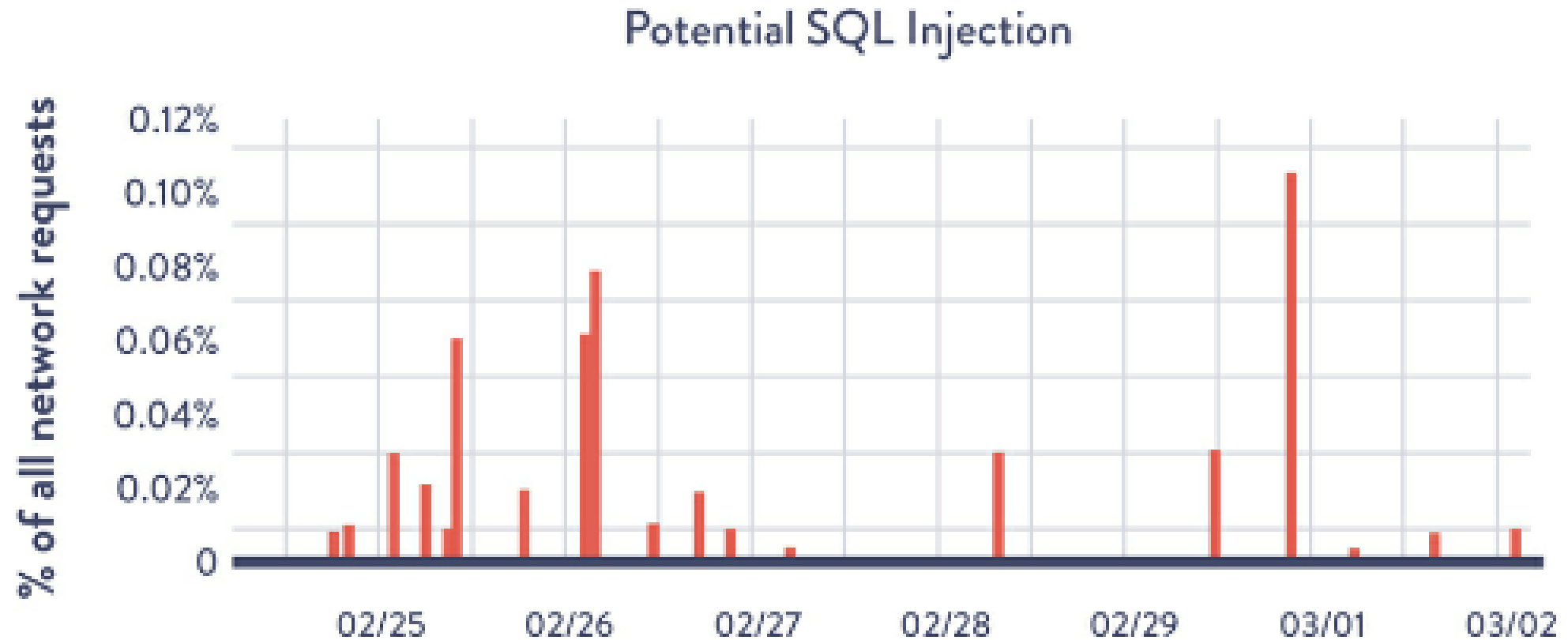
Successful/unsuccessful user login
User password resets
User email address resets
User credit card changes



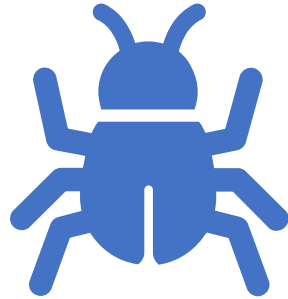
Environment telemetry

OS changes (including build machines)
Security groups changes
Changes to production configurations
Cloud infrastructure changes
Web server errors
XSS attacks
SQLi attacks

Developer seeing SQL Injection Attempts in Graphite at Etsy



Protect Deployment Pipelines



A new surface area vulnerable to attack

Steal code

Inject malicious changes into source code



How to mitigate risks

Automate the creation of build and deploy agents

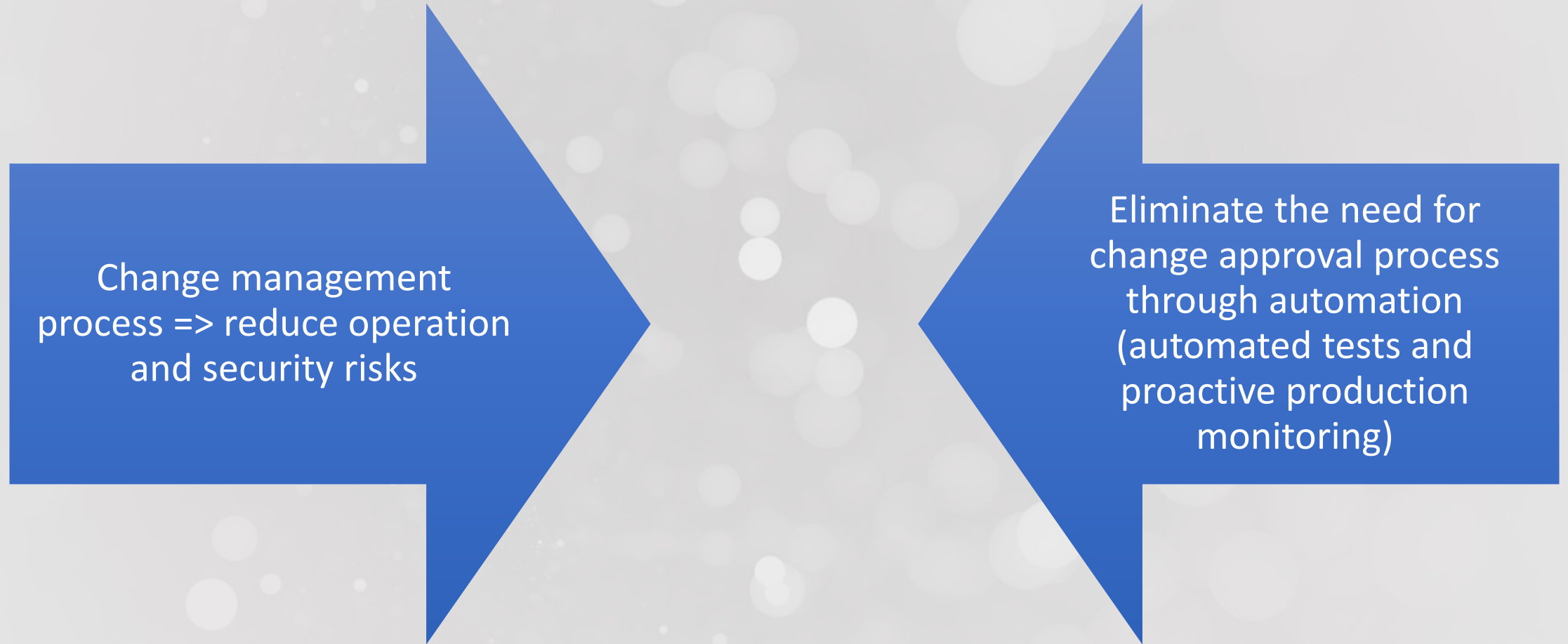
Review all changes introduced in version control

Instrument repository to detect when code contains suspicious API calls

CI process runs on isolated containers or VMs

Read-only access to version-control for CI pipeline

Integrate security and Compliance into Change Approval Process



Type of changes



1

Standard changes – low risk changes that follow an establish pre-approve process (style changes, website content, tax rates, well-understood OS patches, etc.)
– 100% automated



2

Normal changes – high-risk changes that require review and approval, approved by CAB (Change Advisory Board)
– RFC (request for change) – long painful process



3

Urgent changes – emergency / high-risk changes that must be put in production immediately – streamline normal process to work for emergency changes

Recognize Low-Risk Changes as Standard Changes



Reliable deployment pipeline => fast, reliable, undramatic deployments



Demonstrate that changes are low risk enough to be considered standard changes by CAB



Record changes and make them visible



Show high change success rates and low MTTR (mean time to repair)



Build context – link changes to work items (features or production incidents)

What to do When Changes are Normal Changes



Require

Approval from CAB



Ensure

Quick deploys even if not fully automated



Ensure

Submitted changes are complete and accurate (reduce round-trips): context and affected parties



Automate

The creation of RFC (Request For Change): Work item, Build manifest, Tests output



Aim

For winning trust to classify changes as standard

Implement Separation of Duty Through Code Review



Separation of duty = primary control to reduce risk of fraud or mistakes



Changes submitted by **developer** to a code **librarian** for review and approval before deployed by **IT operation**



Traditional approach to implement separation of duties => slows down and reduces the feedback received by engineers (low responsibility and reduced ability to learn)



Alternative: implement separation of duties as a control – pair programming, continuous inspection of code check-ins, and code review

Ensure Documentation and Proof for Auditors and Compliance Officers



Challenge: reduce the gap between DevOps (very little knowledge on the rules) and auditors/compliance officers (very little knowledge on the code)



Solution:

Use telemetry – offers much more visibility
Allow auditors to self-serve



Build controls in deployment pipelines to mitigate risks and demonstrate control effectiveness



Reliance on code review and separation of concerns can leave vulnerabilities – telemetry helps provide necessary visibility



Thank you

